buildandteach

SQL EDITOR · PART 2 QUICK REFERENCE

GROUP BY · HAVING · joins

Same 8 fictional orders + a customers CTE (7 tiers). Free Edition serverless SQL warehouse.
valid = rows with quantity AND unit_price. Baseline: 6 valid orders, 142.50.

1. GROUP BY region, category

```
SELECT region, category, COUNT(*) AS order_count, SUM(order_total) AS revenue
FROM valid GROUP BY region, category ORDER BY region, category;
```

5 groups: E/Books 12 · N/Books 20 · N/Toys 60 (2) · S/Toys 25 · W/Books 25.50 = 6 orders, 142.50.

2. HAVING filters groups

```
... GROUP BY region, category
HAVING SUM(order_total) >= 25
ORDER BY revenue DESC, region, category;
```

3 groups: NORTH/Toys 60.00 · WEST/Books 25.50 · SOUTH/Toys 25.00. WHERE = rows, HAVING = groups.

3. INNER vs LEFT JOIN

```
SELECT v.order_id, v.customer, c.tier, v.order_total
FROM valid v INNER JOIN customers c ON v.customer = c.customer;   -- 5 rows, 130.50
-- LEFT JOIN instead: 6 rows, 142.50 (105 kept, tier NULL)
```

105's customer is NULL, and NULL never matches. Count rows before and after every join.

buildandteach

SQL EDITOR · PART 2 QUICK REFERENCE

CTEs + window functions

4. CTE steps: orders valid by_region

```
by_region AS (SELECT region, COUNT(*) AS order_count, SUM(order_total) AS revenue
  FROM valid GROUP BY region)
SELECT region, order_count, revenue FROM by_region ORDER BY region;
```

EAST 1/12.00 · NORTH 3/80.00 · SOUTH 1/25.00 · WEST 1/25.50.

5. ROW_NUMBER + running total

```
ROW_NUMBER() OVER (PARTITION BY category ORDER BY order_total DESC, order_id DESC)
SUM(order_total) OVER (ORDER BY order_date, order_id
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_revenue
```

Books 103, 101, 105 · Toys 108, 104, 102. Running: 20 → 45 → 70.50 → 100.50 → 112.50 → 142.50.

GROUP BY collapses rows; window functions keep every row and add a column.
Always add a tie-breaker (order_id) so rankings are deterministic.

Practice challenge

LEFT JOIN valid to customers, GROUP BY c.tier: gold 3, silver 1, bronze 1, NULL 1. In the NULL group, COUNT(*) = 1 but COUNT(c.tier) = 0. HAVING COUNT(*) >= 2 leaves only gold (3).

Associate prep: Section 3 aggregations, inner/left joins, gold summaries. No exam guarantee.
Independent education, not affiliated with Databricks. Fictional data. Part 2 of 3.